

# Rapport de projet — NodEX (portefeuille crypto)

Auteur : meranville

Dépôt : mobileapp-2026 — branche **meranville**

Chemin du projet dans le dépôt : project/crypto-wallet/

Date du rapport : mars 2026

---

## 1. Résumé

J'ai développé **NodEX**, une application mobile de type **portefeuille crypto** avec des fonctions financières (soldes, envoi, historique, virements en euros, carte virtuelle côté API), une **authentification** gérée par **Appwrite**, un **assistant conversationnel** via **Groq**, et des paiements intégrés avec **Stripe** (SDK Flutter). Le tout s'appuie sur une **API REST Laravel** exposée sous **/api**. Les **cours des cryptos** affichés à l'écran proviennent de l'**API publique CoinGecko** (appelée depuis l'app Flutter). Ce document présente le travail réalisé, des **captures d'écran**, l'**architecture**, le **fonctionnement des API**, des **extraits de code expliqués**, ce qui manque pour une **mise à disposition du grand public**, et les **sources** qui m'ont aidé.

---

## 2. Contexte et objectifs

- **Contexte** : projet mobile dans le cadre du cours / dépôt partagé L3 (application mobile 2026).
  - **Objectif initial** : disposer d'une application utilisable en **local** (simulateur ou téléphone) connectée à un **backend** documenté.
  - **Périmètre** : prototype avancé / **preuve de concept** orientée fintech, **pas** un produit bancaire agréé.
- 

## 3. Description de la solution

### 3.1 Vue d'ensemble

| Couche       | Technologie                 | Rôle                                                                                       |
|--------------|-----------------------------|--------------------------------------------------------------------------------------------|
| Mobile       | <b>Flutter</b> (Dart)       | Interface, navigation, appels HTTP, intégration Appwrite / Stripe                          |
| Backend      | <b>Laravel</b> (PHP)        | API JSON, virements, chat Groq, données carte / portefeuilles côté serveur                 |
| Auth         | <b>Appwrite</b>             | Inscription, connexion, JWT                                                                |
| IA           | <b>Groq</b>                 | Complétions pour l'assistant (modèle type Llama 3.1 instantané)                            |
| Cours crypto | <b>CoinGecko</b> (API HTTP) | Prix en EUR et variation 24h (appel depuis                                                 |
| Paielements  | <b>Stripe</b>               | <b>WalletProvider</b> )<br>Intégration côté app (clé publique, flux prévus dans le projet) |

### 3.2 Arborescence utile

- flutter\_app/ — code source Flutter (pubspec.yaml, lib/, scripts run\_ios\_sim.sh).
- backend-laravel/ — API Laravel (routes/api.php, contrôleurs, migrations, .env.example).
- docs/rapport-captures/ — captures d'écran jointes à ce rapport (fichiers PNG).
- Documentation complémentaire : README.md (racine du dépôt), STRIPE\_SETUP.md, MIGRATION\_VIREMENTS.md, VIREMENTS\_AUDIT.md, etc.

### 3.3 Dépendances Flutter principales

http, provider, appwrite, flutter\_stripe, flutter\_secure\_storage, local\_auth, shared\_preferences, crypto, qr\_flutter, animations, cupertino\_icons.

---

## 4. Captures d'écran de l'application

Les images suivantes illustrent les principaux écrans de NodEX (simulateur iPhone, mars 2026). Elles sont stockées dans le dépôt sous docs/rapport-captures/.

### 4.1 Connexion

Écran d'accueil non connecté : marque **NodEX**, slogan, onglets de méthode de connexion (email, code, SMS, lien), champs email / mot de passe, lien d'inscription. L'authentification réelle passe par **Appwrite**.

### 4.2 Accueil — tableau de bord

Après connexion : salutation avec le prénom, notifications, carrousel (assistant intelligent, heure locale analogique/numérique), **solde total** en euros, aperçu des portefeuilles (EUR, BTC, etc.), barre d'onglets et bouton flottant **Assistant**.

### 4.3 Accueil — liste des actifs

Liste des cryptos avec **prix en euros**, variation 24h, solde utilisateur ; bloc **Dernières transactions** (virements reçus / envoyés avec pseudonyme) ; bouton **Assistant**.

### 4.4 Portefeuilles (« Mes portefeuilles »)

Carte **valeur totale** (crypto + EUR), raccourcis Recevoir / Envoyer / Swap / Acheter / Historique, liste **Vos actifs** avec adresse, cours et actions.

### 4.5 Historique

Filtres Tout / Envoyés / Reçus / Échanges ; transactions groupées par date ; montants colorés (+ / -) ; navigation par onglets.

### 4.6 Carte virtuelle

Carte type **VISA** avec gradient, **4 derniers chiffres**, titulaire, date d'expiration ; actions **Bloquer** et **Numéro & CVV** (données sensibles gérées avec précaution côté app).

### 4.7 Réglages

Profil (initiale, pseudo, email), **Serveur & assistant (IA)**, Ondes, devise EUR, thème sombre, notifications, sécurité (biométrie, déconnexion tous appareils).



FIGURE 1 – Écran de connexion NodEX



FIGURE 2 – Tableau de bord — accueil



FIGURE 3 – Accueil — actifs et dernières transactions



FIGURE 4 – Écran portefeuilles



FIGURE 5 – Historique des transactions

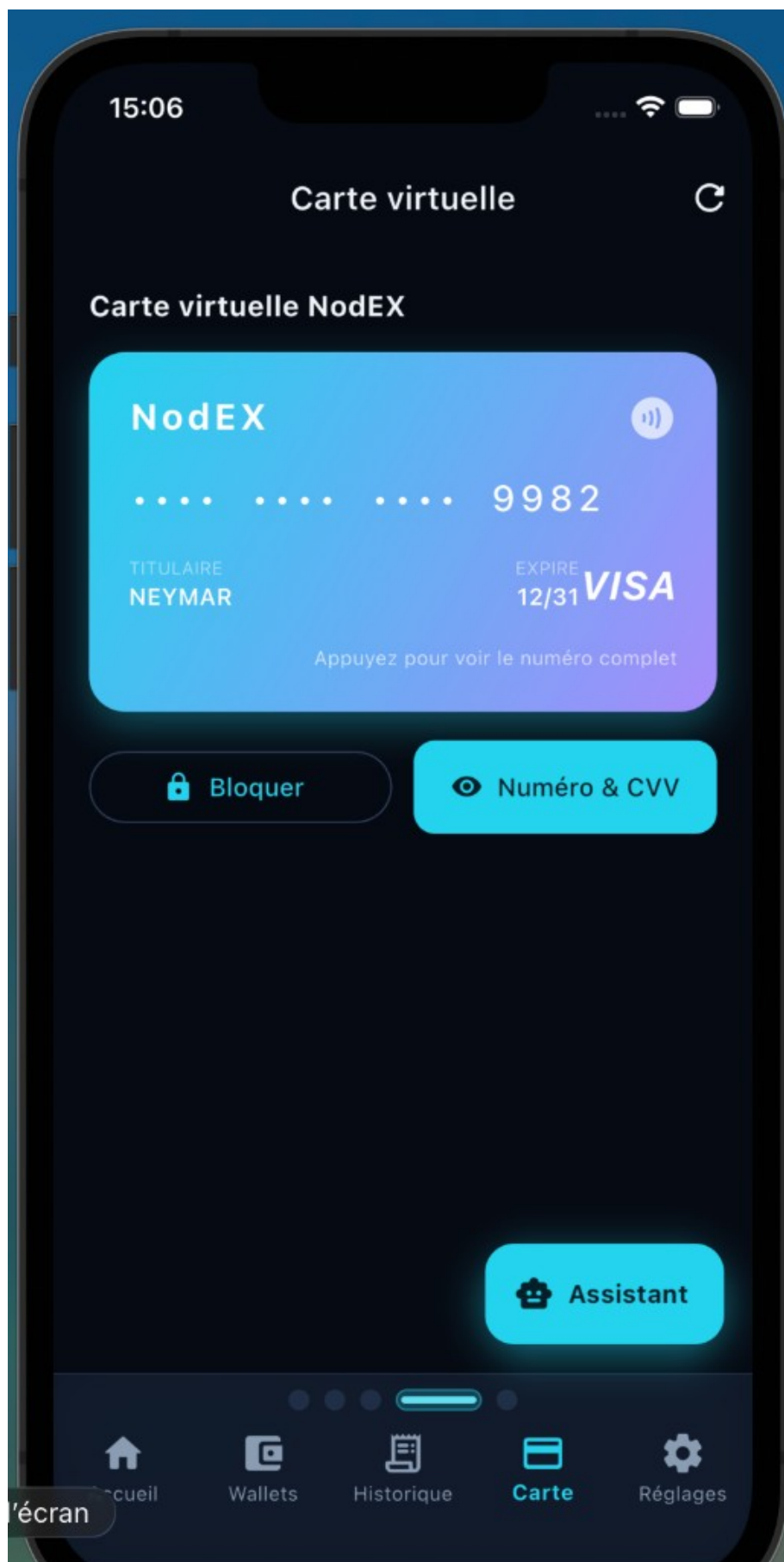


FIGURE 6 – Carte virtuelle



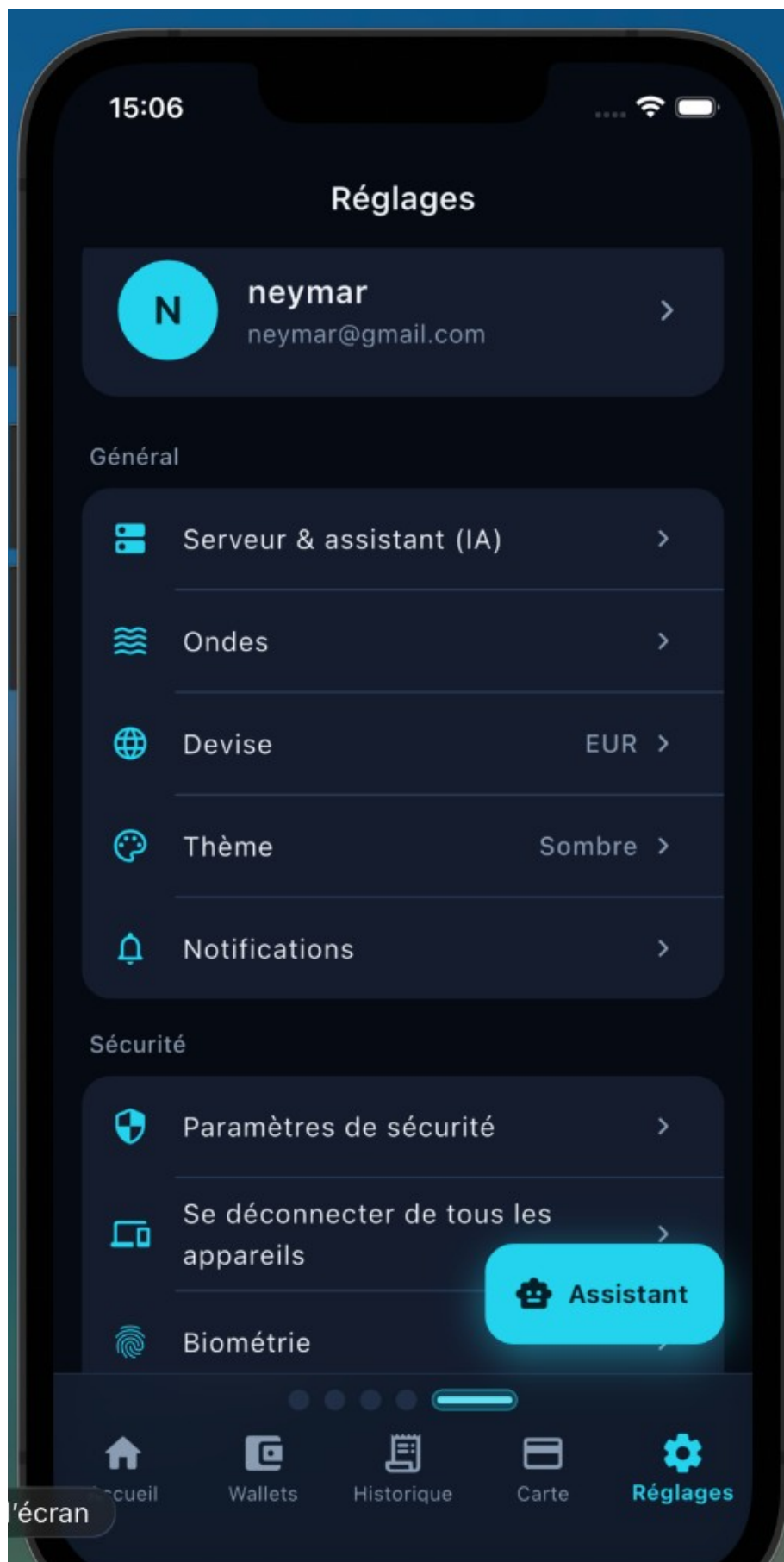


FIGURE 7 – Réglages

## 4.8 Assistant NodEX (IA)

Conversation avec l'assistant : champ de saisie, envoi vers le backend **ou** secours Groq direct selon configuration (voir section 7).

---

## 5. Architecture du projet

### 5.1 Schéma global

L'application suit une architecture **client** — **serveur** : le mobile Flutter consomme l'API Laravel en JSON, s'authentifie via Appwrite, et appelle des services externes (CoinGecko, Groq, Stripe) soit **depuis le téléphone**, soit **via le serveur** (Groq depuis Laravel pour garder la clé secrète et injecter le contexte compte).

flowchart TB

```
subgraph client [Application Flutter]
    UI[Écrans et widgets]
    AP[AuthProvider / SecurityProvider]
    WP[WalletProvider]
    AC[ApiClient HTTP + JWT]
    AW[SDK Appwrite]
    CH[ChatService]
end
```

end

```
subgraph laravel [Backend Laravel]
    R[routes/api.php]
    WC[WalletController]
    VC[VirementController]
    CC[ChatController]
    CD[CardController]
    DB[(Base de données)]
end
```

end

```
subgraph externe [Services externes]
    APPW[(Appwrite Cloud)]
    CG[CoinGecko API]
    GQ[Groq API]
    ST[Stripe]
end
```

end

UI --> AP

UI --> WP

WP --> AC

WP --> CG

CH --> AC

CH --> GQ

AC --> R

R --> WC

R --> VC

R --> CC

R --> CD

WC --> DB

VC --> DB

CC --> DB

CC --> GQ

CD --> DB

AP --> AW

AW --> APPW

UI --> ST



FIGURE 8 – Assistant NodEX  
11

## 5.2 Navigation dans l'app Flutter

Le fichier `lib/app.dart` définit la logique d'affichage :

1. **Splash** d'ouverture (`NodexOpeningSplash`).
2. Si chargement auth → `LoadingScreen`.
3. Si récupération mot de passe → `UpdatePasswordScreen`.
4. Si email non vérifié → `EmailVerificationPendingScreen`.
5. Si non connecté → `LoginScreen`.
6. Si verrouillage PIN / biométrie → `AppLockScreen`.
7. Sinon → **HomeTabs** : cinq onglets (`DashboardScreen`, `WalletsScreen`, `HistoryScreen`, `CardScreen`, `SettingsScreen`) avec `PageTransitionSwitcher` + `BottomNavigationBar`, fond `FuturisticBackground`, et un **FAB « Assistant »** qui ouvre `ChatScreen`.

## 5.3 Couches côté Laravel

- **Routes** : `routes/api.php` enregistre les URL sous le préfixe `/api` (souvent monté par `bootstrap/app.php`).
- **Contrôleurs** : logique métier (virements, carte, chat, wallets).
- **Trait ResolvesNodexUser** : lit le JWT Bearer, extrait l'identifiant Appwrite, charge la ligne `NodexUser` en base.
- **Modèles** : `NodexUser`, `VirementEur`, `UserCard`, etc.

---

## 6. API REST — liste et fonctionnement

Toutes les URL ci-dessous sont relatives à la **base API** configurée dans l'app, par exemple `http://127.0.0.1:8000/api`. Les routes **protégées** attendent un en-tête :

`Authorization: Bearer <JWT Appwrite>`.

| Méthode | Chemin                   | Rôle                                                                                                                     |
|---------|--------------------------|--------------------------------------------------------------------------------------------------------------------------|
| GET     | <code>/health</code>     | Vérifier que le serveur répond (ok, horodatage).                                                                         |
| GET     | <code>/debug/auth</code> | <b>Debug uniquement</b> : indique si un Bearer est présent et décode le payload JWT (à <b>supprimer en production</b> ). |
| GET     | <code>/wallets</code>    | Liste des portefeuilles crypto (ETH, BTC, SOL, ALGO) : symbole, blockchain, adresse dérivée, solde (cache serveur ou 0). |
| GET     | <code>/card</code>       | Détail carte virtuelle (création si besoin) : numéro formaté, last4, expiration, CVV, PIN, titulaire.                    |

| Méthode | Chemin             | Rôle                                                                                                                                                                                                                                     |
|---------|--------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| POST    | /chat/groq         | Corps JSON : <b>messages</b> , optionnellement <b>model</b> , <b>temperature</b> , <b>max_tokens</b> . Le serveur enrichit le contexte système avec solde EUR, IBAN, pseudonyme, cryptos, derniers virements, puis appelle <b>Groq</b> . |
| GET     | /virements/balance | Solde euro du compte NodEX ( <b>balanceEur</b> ).                                                                                                                                                                                        |
| GET     | /virements/me      | Infos bancaires synthétiques : solde, IBAN, pseudonyme, titulaire.                                                                                                                                                                       |
| GET     | /virements/history | Historique fusionné envoyés/reçus (limite côté serveur).                                                                                                                                                                                 |
| POST    | /virements/send    | Envoi d'un virement interne (montant, destinataire identifié par pseudonyme ou id selon implémentation du contrôleur).                                                                                                                   |

### 6.1 Chaîne d'authentification JWT → utilisateur Laravel

1. L'utilisateur se connecte avec **Appwrite** dans Flutter.
2. L'app récupère un **JWT** (ou token stocké en secours dans **SharedPreferences**).
3. **ApiClient** ajoute **Authorization: Bearer ...** à chaque requête vers Laravel.
4. Le trait **ResolvesNodexUser** decode le **payload** du JWT (deuxième segment en base64), lit **userId**, **sub** ou champs imbriqués, puis fait **NodexUser::where('appwriteId', ...)->first()**.
5. Si aucune ligne : réponses **401** ou message d'erreur métier.

### 6.2 Flux assistant IA (Groq)

- **Mode recommandé** : Flutter envoie POST /api/chat/groq → Laravel ajoute les **données réelles du compte** dans le message système (pour limiter les inventions du modèle) → Laravel appelle <https://api.groq.com/openai/v1/chat/completions> avec **GROQ\_API\_KEY** dans **.env**.
- **Mode secours** : si le serveur est indisponible, sans clé, ou erreur réseau, **ChatService** peut appeler Groq **directement** depuis l'app si une clé est saisie dans les réglages (moins idéal pour la confidentialité de la clé).

### 6.3 Prix des cryptomonnaies

- **Pas** calculés par Laravel dans **WalletController** pour l'affichage marché.
- Le **WalletProvider** (Flutter) appelle **CoinGecko** (**simple/price** en EUR + **include\_24hr\_change**) pour remplir **\_prices** et **\_changes24h**, puis combine avec les soldes renvoyés par GET /api/wallets.

## 7. Explication de code clé

### 7.1 ApiClient — client HTTP unique avec JWT

Fichier : flutter\_app/lib/services/api\_client.dart.

- **Singleton** (`factory ApiClient()`) : une seule instance pour toute l'app.
- `_getToken()` : priorité au token **Appwrite** via `AuthServiceAppwrite`, sinon valeur stockée localement (`jwt_token`).
- `_headers()` : ajoute `Content-Type: application/json` et `Authorization: Bearer ...` si demandé.
- `get / post` : construisent l'URL avec `ApiConfig.baseUrl` + chemin (ex. `/virements/history`), timeout 15 s. Un **401** ne vide pas brutalement la session : le commentaire indique qu'on peut resynchroniser le JWT avec Appwrite.

### 7.2 ApiConfig — URL de l'API

Fichier : flutter\_app/lib/config/api\_config.dart.

- Ordre : **URL enregistrée dans Réglages** → variable de compilation `API_BASE_URL` → défauts (127.0.0.1:8000/api sur iOS/desktop, 10.0.2.2 sur émulateur Android).
- `_normalizeApiBase` : garantit que l'URL se termine par `/api`.

### 7.3 ResolvesNodexUser — qui est l'utilisateur côté API ?

Fichier : backend-laravel/app/Http/Controllers/Concerns/ResolvesNodexUser.php.

- Découpe le header `Authorization`, extrait le JWT, décode le payload JSON.
- Cherche l'identifiant Appwrite dans `userId`, `sub`, `$id`, ou sous-objet `user`.
- Retourne le modèle **NodexUser** lié à `appwriteId`. Tous les contrôleurs métier utilisent ce trait pour rester cohérents.

### 7.4 ChatController — proxy Groq + contexte compte

Fichier : backend-laravel/app/Http/Controllers/ChatController.php.

- Vérifie l'utilisateur et la présence de `GROQ_API_KEY`.
- `injectLiveAccountContext` : fusionne un bloc texte (solde EUR, nom, pseudo, IBAN, soldes crypto depuis le cache, derniers virements) dans le message **system** envoyé au modèle.
- Objectif documenté dans le code : l'IA ne doit **pas inventer** soldes ou IBAN pour ce compte.

### 7.5 VirementController — virements internes EUR

Fichier : backend-laravel/app/Http/Controllers/VirementController.php.

- `balance / me` : lecture du solde et des infos affichables type RIB.
- `history` : fusionne envois et réceptions, résout les **pseudonymes** des contreparties, trie par date.
- `send` : transaction en base (débit/crédit) sous contraintes de solde (suite du fichier non citée ici ligne par ligne).

## 7.6 WalletController — portefeuilles crypto

Fichier : `backend-laravel/app/Http/Controllers/WalletController.php`.

- Pour chaque symbole (ETH, BTC, SOL, ALGO), construit un objet avec **adresse déterministe** (hash SHA256 du couple `userId` + symbole avec préfixe `bc1/0x` selon la chaîne).
- Les **soldes** viennent du **cache** Laravel `nodex_crypto_{userId}` ou sont à zéro par défaut.

## 7.7 CardController — carte virtuelle

Fichier : `backend-laravel/app/Http/Controllers/CardController.php`.

- Si aucune ligne `UserCard` : génération numéro (avec contrôle type **Luhn** dans la suite du fichier), date d'expiration, CVV, PIN, association au `userId`.

## 7.8 ChatService — stratégie double (Laravel puis direct)

Fichier : `flutter_app/lib/services/chat_service.dart`.

- Construit l'historique des messages (limite des derniers tours).
- Tente d'abord `POST /chat/groq` via `ApiClient`.
- Si **401**, **503**, erreurs serveur ou **réseau**, tente **Groq direct** avec `GroqDirectConfig.effectiveKey` et un message système incluant éventuellement un **contexte local** passé depuis l'UI.

## 7.9 WalletProvider — agrégation CoinGecko + API + virements

Fichier : `flutter_app/lib/providers/wallet_provider.dart`.

- `_fetchPricesFromCoinGecko` : requête GET vers l'API CoinGecko, met à jour prix EUR et variation 24h.
- `fetch` : charge les wallets via l'API Laravel et fusionne l'**historique des virements** dans la liste des transactions affichées sur l'accueil et l'historique (`_mergeVirementsIntoTransactions`).

---

## 8. Méthode et mise en œuvre

1. Mise en place du backend Laravel (`.env`, Composer, migrations, `php artisan serve`).
2. Développement de l'app Flutter et connexion à l'API (`API_BASE_URL` via `--dart-define` ou réglages in-app).
3. Configuration Appwrite et variables sensibles **hors dépôt** (fichier `.env` ignoré par Git).
4. Tests manuels sur simulateur iOS / appareil réel (réseau local, URL du Mac).
5. Versionnement Git et push sur la branche **meranville**.

---

## 9. Ce qui manque pour un déploiement « grand public »

Ce projet est adapté au **développement et à la démonstration**. Pour une **publication large** (stores, utilisateurs inconnus, charge réelle), il faudrait notamment :

### 9.1 Produit et conformité

- **Statut légal** : services de paiement, crypto et « banking » sont **réglementés** (ex. Europe : DSP2, AML, licences selon l'activité). Un prototype étudiant **ne remplace pas** un cadre juridique ni un établissement

agréé.

- **CGU / politique de confidentialité / RGPD** : textes validés, base légale du traitement des données, droits utilisateurs, DPO si nécessaire.
- **Mentions « démo / non régulé »** si l'app reste un projet pédagogique accessible publiquement.

## 9.2 Sécurité

- Supprimer **/api/debug/auth** et tout endpoint de debug en production.
- **HTTPS** obligatoire partout (API, Appwrite, callbacks).
- Secrets uniquement en variables d'environnement **serveur** ; rotation des clés ; principe du moindre privilège.
- Revue des en-têtes CORS, limitation du débit (**rate limiting**), validation **cryptographique** du JWT (au-delà du simple décodage base64 utilisé pour résoudre l'utilisateur — en production il faut vérifier la signature avec les clés Appwrite).
- Audit de sécurité (OWASP Mobile / API) et tests d'intrusion pour un vrai lancement.

## 9.3 Infrastructure

- Hébergement **géré** (serveur, base de données, sauvegardes, monitoring, logs centralisés).
- **CI/CD** (tests automatiques, build signé iOS/Android).
- **Scalabilité** (file d'attente pour tâches longues, cache si besoin).

## 9.4 Qualité et maintenance

- **Tests automatisés** (unitaires, intégration API, tests widget Flutter).
- Gestion des **versions**, changelog, canal de **support** utilisateur.
- **Accessibilité** (a11y) et **internationalisation** (i18n) si public multilingue.

## 9.5 Stores (Apple App Store, Google Play)

- Comptes développeur payants, fiches conformes aux guidelines, captures d'écran, politique de confidentialité hébergée en URL publique.
- Déclaration des **achats intégrés** / Stripe selon les règles de chaque store.
- Build **signé**, provisioning iOS, Android App Bundle.

## 9.6 Opérations financières réelles

- Compte **Stripe** (ou autre PSP) en mode **live**, webhooks sécurisés, conformité **PCI** (ne jamais stocker PAN en clair).
- KYC / lutte contre la fraude si encaissement ou comptes utilisateurs réels.

En résumé : le travail actuel constitue une **base technique solide pour un projet universitaire** ; le passage au **grand public** implique juridique, sécurité, exploitation et qualité logicielle bien au-delà du périmètre d'un cours.



## 10. Difficultés rencontrées (exemples types)

- Faire coïncider l'**URL de l'API** entre simulateur, téléphone physique et machine hébergeant Laravel (adresse IP locale, pare-feu).
- Gérer l'**authentification** (JWT Appwrite) côté Laravel et côté Flutter de manière cohérente.
- Comprendre la **structure du dépôt** (Flutter dans un sous-dossier, pas à la racine).
- Configurer **Stripe** et **Appwrite** sans commiter de secrets (`.env`, `.gitignore`).
- Limiter les **hallucinations** de l'IA sur les soldes grâce au **contexte injecté** côté Laravel.

(À adapter avec mes propres anecdotes pour l'oral.)

---

## 11. Conclusion

NodEX regroupe une **application Flutter**, une **API Laravel**, **Appwrite**, **Groq**, **CoinGecko** et **Stripe** dans une expérience de portefeuille et de services associés. Le rendu est **démontrable en local**, illustré par des **captures d'écran** dans ce rapport, et documenté dans le `README.md` du dépôt. Pour une **diffusion publique**, il reste indispensable de traiter **légal**, **sécurité**, **hébergement**, **stores** et **exploitation** comme un produit à part entière.

---

## 12. Sources, documentation et ressources utilisées

Les liens ci-dessous sont les **sources officielles ou principales** qui m'ont aidé. Les URL sont données en entier pour faciliter la vérification.

### 12.1 Frameworks et langages

| Ressource               | URL                                                                     |
|-------------------------|-------------------------------------------------------------------------|
| Flutter — documentation | <a href="https://docs.flutter.dev/">https://docs.flutter.dev/</a>       |
| Dart — langage          | <a href="https://dart.dev/">https://dart.dev/</a>                       |
| Laravel — documentation | <a href="https://laravel.com/docs">https://laravel.com/docs</a>         |
| PHP                     | <a href="https://www.php.net/docs.php">https://www.php.net/docs.php</a> |
| Composer (PHP)          | <a href="https://getcomposer.org/doc/">https://getcomposer.org/doc/</a> |

### 12.2 Services et API externes du projet

| Ressource                                   | URL                                                                                                 |
|---------------------------------------------|-----------------------------------------------------------------------------------------------------|
| Appwrite — documentation                    | <a href="https://appwrite.io/docs">https://appwrite.io/docs</a>                                     |
| Stripe — documentation développeurs         | <a href="https://stripe.com/docs">https://stripe.com/docs</a>                                       |
| Stripe — Flutter SDK (flutter_stripe)       | <a href="https://pub.dev/packages/flutter_stripe">https://pub.dev/packages/flutter_stripe</a>       |
| Groq — console / clés API                   | <a href="https://console.groq.com/">https://console.groq.com/</a>                                   |
| Groq — documentation modèles API            | <a href="https://console.groq.com/docs/models">https://console.groq.com/docs/models</a>             |
| CoinGecko — documentation API (prix crypto) | <a href="https://www.coingecko.com/en/api/documenta">https://www.coingecko.com/en/api/documenta</a> |

### 12.3 Packages Flutter (référence pub.dev)

| Package  | URL                                                                               |
|----------|-----------------------------------------------------------------------------------|
| provider | <a href="https://pub.dev/packages/provider">https://pub.dev/packages/provider</a> |
| http     | <a href="https://pub.dev/packages/http">https://pub.dev/packages/http</a>         |

| Package                | URL                                                                                                           |
|------------------------|---------------------------------------------------------------------------------------------------------------|
| appwrite (SDK Dart)    | <a href="https://pub.dev/packages/appwrite">https://pub.dev/packages/appwrite</a>                             |
| flutter_secure_storage | <a href="https://pub.dev/packages/flutter_secure_storage">https://pub.dev/packages/flutter_secure_storage</a> |
| local_auth             | <a href="https://pub.dev/packages/local_auth">https://pub.dev/packages/local_auth</a>                         |
| shared_preferences     | <a href="https://pub.dev/packages/shared_preferences">https://pub.dev/packages/shared_preferences</a>         |
| qr_flutter             | <a href="https://pub.dev/packages/qr_flutter">https://pub.dev/packages/qr_flutter</a>                         |
| animations             | <a href="https://pub.dev/packages/animations">https://pub.dev/packages/animations</a>                         |
| flutter_lints          | <a href="https://pub.dev/packages/flutter_lints">https://pub.dev/packages/flutter_lints</a>                   |

## 12.4 UI / design

| Ressource                                       | URL                                                                               |
|-------------------------------------------------|-----------------------------------------------------------------------------------|
| Material Design 3                               | <a href="https://m3.material.io/">https://m3.material.io/</a>                     |
| Panache (thème Flutter, référence dans le code) | <a href="https://github.com/rxlabz/panache">https://github.com/rxlabz/panache</a> |

## 12.5 Outils, versionnement, présentation

| Ressource                                          | URL                                                           |
|----------------------------------------------------|---------------------------------------------------------------|
| Git — documentation                                | <a href="https://git-scm.com/doc">https://git-scm.com/doc</a> |
| GitHub                                             | <a href="https://github.com/">https://github.com/</a>         |
| Gamma (présentations assistées par IA, si utilisé) | <a href="https://gamma.app/">https://gamma.app/</a>           |

## 12.6 Sécurité et bonnes pratiques (références générales)

| Ressource                              | URL                                                                                                         |
|----------------------------------------|-------------------------------------------------------------------------------------------------------------|
| OWASP Mobile Top 10                    | <a href="https://owasp.org/www-project-mobile-top-10/">https://owasp.org/www-project-mobile-top-10/</a>     |
| CNIL — RGPD (information grand public) | <a href="https://www.cnil.fr/fr/rgpd-de-quoi-parle-t-on">https://www.cnil.fr/fr/rgpd-de-quoi-parle-t-on</a> |

## 12.7 Dépôt du cours / du projet

| Ressource                   | URL                                                                                                   |
|-----------------------------|-------------------------------------------------------------------------------------------------------|
| Dépôt GitHub mobileapp-2026 | <a href="https://github.com/roorkilla/mobileapp-2026">https://github.com/roorkilla/mobileapp-2026</a> |

## 13. Déclaration d'honnêteté académique

Je déclare que ce rapport décrit mon travail et que les sources externes sont citées dans la section 12. Les captures d'écran proviennent de mon exécution de l'application NodEX. Les extraits de code tiers respectent les licences des projets concernés.